

INVESTIGACIÓN: ARQUITECTURA MDI EN JAVA SWING

INTRODUCCIÓN

La arquitectura MDI (Multiple Document Interface) es una forma de organizar aplicaciones de escritorio donde existe una ventana principal que contiene diferentes ventanas secundarias. En Java Swing se puede implementar utilizando componentes como JFrame, JDesktopPane, JInternalFrame y JMenuBar. Es común en sistemas administrativos, de ventas e inventarios.

1. ¿QUÉ ES UN FORMULARIO CONTENEDOR (MDI)?

MDI significa Multiple Document Interface o Interfaz de Múltiples Documentos. Consiste en tener una ventana principal que funciona como contenedor de diferentes formularios hijos.

Por ejemplo, un sistema puede tener módulos de Clientes, Productos, Ventas y Reportes. Todos estos módulos pueden abrirse dentro de la misma ventana principal.

En Java, normalmente se utiliza:

- * JFrame: ventana principal.
- * JDesktopPane: área donde se colocan los formularios.
- * JInternalFrame: formularios hijos.
- * JMenuBar: barra de menús.

2. ¿QUÉ ES UN JDESKTOPPANE?

JDesktopPane es un componente de Java Swing que funciona como un área de trabajo o escritorio dentro de la ventana principal. Su función es contener los diferentes JInternalFrame.

Ejemplo:

```
JDesktopPane escritorio = new JDesktopPane();
```

Después se pueden agregar formularios:

```
escritorio.add(formulario);
```

Por lo tanto, el JDesktopPane puede entenderse como el contenedor donde se muestran las diferentes ventanas internas de la aplicación.

3. ¿QUÉ ES UN JINTERNALFRAME?

JInternalFrame representa una ventana secundaria dentro de un JDesktopPane. Puede utilizarse para crear formularios como Clientes, Productos o Ventas.

Ejemplo:

```
JInternalFrame clientes = new JInternalFrame(
"Clientes",
true,
true,
true,
true
);
```

```
escritorio.add(clientes);
clientes.setVisible(true);
```

Un JInternalFrame puede contener componentes como botones, cajas de texto, tablas, etiquetas y otros elementos de Java Swing.

La diferencia principal con JFrame es que el JInternalFrame funciona dentro de la aplicación principal y no como una ventana independiente del sistema operativo.

4. ¿CÓMO FUNCIONA UN JMENUBAR?

JMenuBar permite crear la barra de menús de una aplicación. Normalmente se encuentra en la parte superior de la ventana principal.

Ejemplo:

```
JMenuBar barra = new JMenuBar();

JMenu archivo = new JMenu("Archivo");

JMenuItem clientes = new JMenuItem("Clientes");

archivo.add(clientes);
barra.add(archivo);

setJMenuBar(barra);
```

El JMenuBar puede contener diferentes menús y opciones para acceder a los módulos de la aplicación.

5. ¿CÓMO ABRIR FORMULARIOS HIJOS DESDE UN MENÚ?

Para abrir un formulario desde un menú se utiliza un evento. Por ejemplo:

```
clientes.addActionListener(e -> abrirClientes());
```

El método puede crear un JInternalFrame y agregarlo al JDesktopPane:

```
private void abrirClientes() {
```

```

...
JInternalFrame ventana =
    new JInternalFrame("Clientes", true, true, true, true);

ventana.setSize(500, 300);

escritorio.add(ventana);

ventana.setVisible(true);
...

}

```

El proceso es:

Usuario selecciona "Clientes" → JMenuitem detecta el clic → se ejecuta el evento → se crea el JInternalFrame → se agrega al JDesktopPane → se muestra el formulario.

6. EJEMPLO DE LA ESTRUCTURA MDI

Una aplicación MDI puede tener la siguiente estructura:

```

JFrame
|
+-- JMenuBar
|   |
|   +-- Clientes
|   +-- Productos
|   +-- Ventas
|   +-- Reportes
|
+-- JDesktopPane
|
+-- JInternalFrame: Clientes
+-- JInternalFrame: Productos
+-- JInternalFrame: Ventas

```

De esta manera, el JFrame es la ventana principal, el JMenuBar permite navegar por el sistema, el JDesktopPane es el área de trabajo y los JInternalFrame son los formularios hijos.

7. VENTAJAS DE UTILIZAR ESTA ARQUITECTURA

La arquitectura MDI presenta varias ventajas:

- * Permite organizar los diferentes módulos dentro de una sola aplicación.
- * Facilita la navegación mediante menús.

- * Permite tener varios formularios abiertos al mismo tiempo.
- * Centraliza elementos como menús y barras de herramientas.
- * Facilita la división de una aplicación en diferentes módulos.
- * Es adecuada para sistemas administrativos, de ventas, inventarios y facturación.

Por ejemplo, un usuario puede tener abiertos los formularios de Clientes y Productos mientras trabaja en el formulario de Ventas, sin necesidad de abrir diferentes programas.

8. DESVENTAJAS

Una desventaja es que demasiadas ventanas internas pueden hacer que la interfaz sea difícil de organizar. Por eso, es recomendable controlar qué formularios pueden estar abiertos y evitar que el mismo formulario se abra varias veces.

CONCLUSIÓN

La arquitectura MDI permite desarrollar aplicaciones de escritorio organizadas mediante una ventana principal que contiene diferentes formularios hijos.

En Java Swing, JFrame funciona como ventana principal, JDesktopPane como área de trabajo, JInternalFrame como formulario hijo y JMenuBar como sistema de navegación.

Esta arquitectura facilita la organización y navegación de sistemas con varios módulos, por lo que puede ser una buena alternativa para desarrollar aplicaciones administrativas y de gestión.

BIBLIOGRAFÍA

Oracle. Java Documentation. JDesktopPane Class.

Oracle. Java Documentation. JInternalFrame Class.

Oracle. Java Documentation. JMenuBar Class.

Oracle. Java Documentation. How to Use Internal Frames. Java Swing Tutorial.

Oracle. Java Documentation. How to Use Menus. Java Swing Tutorial.